ProTIP: Progressive Tool Retrieval Improves Planning

Raviteja Anantha*, Bortik Bandyopadhyay*, Anirudh Kashi, Sayantan Mahinder, Andrew W Hill, Srinivas Chappidi
Apple

arXiv:2312.10332v1 [cs.IR] 16 Dec 2023

Abstract

Large language models (LLMs) are increasingly employed for complex multi-step planning tasks, where the tool retrieval (TR) step is crucial for achieving successful outcomes. Two prevalent approaches for TR are single-step retrieval, which utilizes the complete query, and sequential retrieval using task decomposition (TD), where a full query is segmented into discrete atomic subtasks. While single-step retrieval lacks the flexibility to handle "inter-tool dependency," the TD approach necessitates maintaining "subtask-tool atomicity alignment," as the toolbox can evolve dynamically. To address these limitations, we introduce the **Pro**gressive **T**ool retrieval to **I**mprove **P**lanning (ProTIP) framework. ProTIP is a lightweight, contrastive learning-based framework that implicitly performs TD without the explicit requirement of subtask labels, while simultaneously maintaining subtask-tool atomicity. On the ToolBench dataset, ProTIP outperforms the ChatGPT task decomposition-based approach by a remarkable margin, achieving a 24% improvement in Recall@K=10 for TR and a 41% enhancement in tool accuracy for plan generation.

1 Introduction

Large language models (LLMs) (Brown et al., 2020; Chowdhery et al., 2022; Ouyang et al., 2022; Zhang et al., 2022; Zeng et al., 2022; OpenAI, 2023b; Touvron et al., 2023; Bubeck et al., 2023; Thoppilan et al., 2022) have witnessed remarkable advancements in recent years, driven by efficient pre-training on massive text datasets and the application of sophisticated algorithms like reinforcement learning from human feedback (RLHF) (Ouyang et al., 2022) to better align these models with human preferences. This progress has unlocked emergent capabilities (Wei et al., 2022a) in LLMs, which can be further refined to enhance their instruction-following abilities (Taori et al., 2023; Chiang et al., 2023). Additionally, LLMs have demonstrated the potential for in-context learning (Brown et al., 2020; Xie et al., 2021; Min et al., 2022) and chain-of-thought prompting (Wei et al., 2022b; Kojima et al., 2022; Wang et al., 2022), expanding their applicability across a wide spectrum of application domains.

Harnessing the power of LLMs as language understanding agent (Shen et al., 2023a) to tackle complex tasks has emerged as a burgeoning research area. This endeavor presents a challenge due to the inherent complexity of multi-step planning (Huang et al., 2022; Ahn et al., 2022; Singh et al., 2022). To address this challenge, we employ a flexible planning framework that seamlessly integrates an LLM with an external toolbox containing application specific atomic actions. The LLM planner bridges the gap between natural language instructions and executable actions by effectively selecting appropriate APIs/tools from a curated list presented in the LLM prompt. These tools are retrieved using specialized techniques from the external toolbox (Schick et al., 2023; Qin et al., 2023a; Patil et al., 2023; Qin et al., 2023b; Shen et al., 2023a). The terms tool and API are used interchangeably throughout this paper.

Within multi-step planning framework with an external toolbox, the tool retrieval (TR) step plays a crucial role in determining the overall planner's performance. The TR step can be implemented either as a single-step process utilizing the entire query or as an iterative approach that decomposes the query into individual atomic subtasks (Khot et al., 2022; Wu et al., 2023). The single-step TR approach, however, is unable to handle "inter-tool dependency" in multi-step execution scenarios. This limitation becomes evident, for instance, when selecting between tool-A and tool-B, where the choice depends on the successful execution

*Equal contributions.

of a previously chosen tool. In contrast, the TD-based TR approach necessitates maintaining the alignment between the exact subtask in question and the appropriate tool to be used from the employed toolbox version, thus creating a "subtask-tool atomicity alignment," problem when training the planner. This dependency often requires either frequent fine-tuning of lightweight TD models or the utilization of an LLM, such as ChatGPT (OpenAI, 2023a), for TD. Furthermore, both these approaches operate within the text space, making them susceptible to various issues such as "out of vocabulary" tokens, which can hinder accurate semantic representation of the subtasks and ultimately impact the planner's performance.

To overcome these limitations, we introduce the **Pro**gressive **T**ool retrieval to **I**mprove **P**lanning (ProTIP) framework. Our TR strategy draws inspiration from advancements in the word embedding literature, where prior works (Mikolov et al., 2013a; Pennington et al., 2014; Levy and Goldberg, 2014) have shown the effectiveness of representing semantic relationships between words by embedding them in a vector space. Extending this concept to complex queries and tools, we leverage task-specific fine-tuning to achieve our progressive TR requirements.

ProTIP initially encodes the given query and tool descriptions to minimize the Euclidean distance between relevant tools corresponding to the first subtask and the query in a semantic space, without explicitly performing task decomposition. Subsequently, the ProTIP module iteratively transforms the query embedding by subtracting previously retrieved tool description embedding from the query embedding. The resultant embedding from this iterative subtraction aligns more closely in semantic space with a natural language query formed by eliminating previously executed subtasks from the full query, while focusing on the next most important subtask to be executed out of the remaining ones. ProTIP is fine-tuned using contrastive loss to learn embeddings with above-mentioned characteristics, more details in section 3.1. As a consequence, ProTIP provides flexibility by enabling incremental TR, while incorporating execution history (e.g., the previously selected tool and execution result) without the overhead of maintaining atomicity for the TD. The contributions of this work are as follows:

- We introduce ProTIP, a novel progressive TR framework, that efficiently performs TR for complex requests involving inter-subtask dependencies, while factoring in the execution history, without requiring explicit TD.
- We comprehensively compare various TR methods and their impact on LLM-powered planning agents using the public ToolBench dataset (Qin et al., 2023b).
- To the best of our knowledge, we are the first to establish that lightweight (non-LLM) fine-tuning based tool retrieval approaches, such as ProTIP, can outperform state-of-the-art LLM-augmented approaches, such as ChatGPT-based TD for TR.

2 Data

We evaluate the efficacy of ProTIP-based LLM-Planner in generating step-by-step plans using the ToolBench (Qin et al., 2023b) dataset, one of the most extensive instruction-tuning dataset for tool utilization, encompassing 16,000 tools and 49 categories. ToolBench includes 27,000 complex requests, each of which may require the use of one or more APIs/Tools to complete subtasks in a specific order. Each request is accompanied by tool and plan labels, which represent a series of more granular step-by-step instructions.

Figure 1 illustrates the distribution of the number of tools required for each query, providing insights into the complexity of requests within the dataset. The analysis reveals that the maximum number of subtasks in a query is 6. This information is utilized to establish the upper bound for top-k in TR and planner experiments, as detailed in section 3.2.

Figure 2 shows the distribution of input token length to the planner in train and test sets. Notably, 12.25% (97 data points) of the test set and 12.30% (9,070 data points) of the training set exceed the context window size of 2048. This substantial proportion of lengthy inputs is expected to cause truncation, potentially hindering the model's ability to achieve optimal performance.

Figure 3 depicts the distribution of ground truth tool IDs in the dataset. Notably, a significant proportion of tool IDs fall within the range of 0 to 2000 for both the training and test sets. This imbalance in tool representation could potentially bias the model towards these more frequently appearing tools.

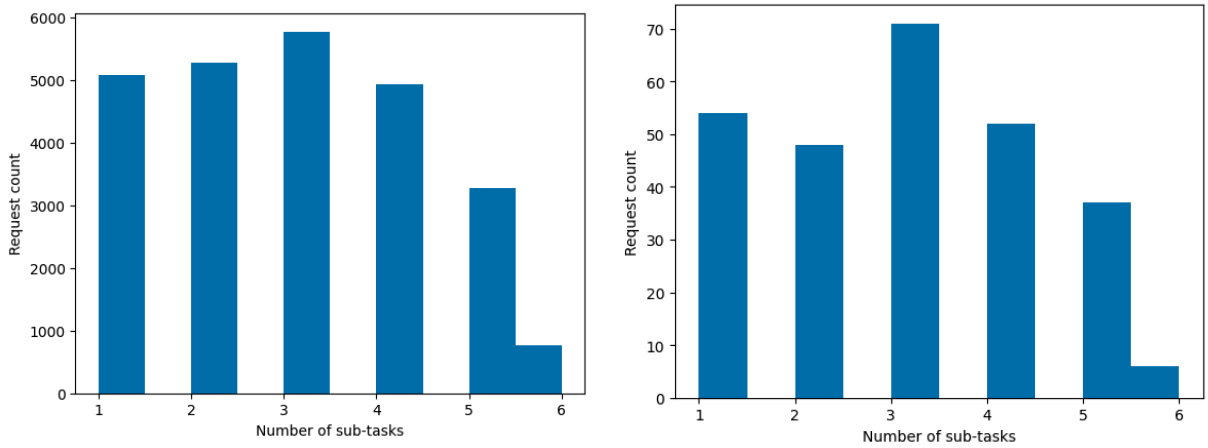

Figure 1: Distribution of requests in the ToolBench training (left) and test (right) sets according to the number of subtasks involved in each request.

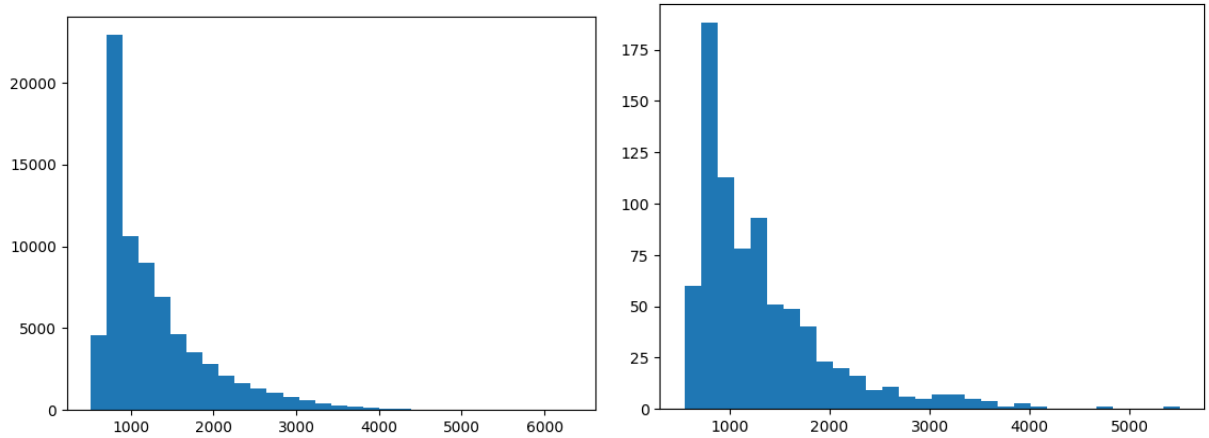

Figure 2: Distributions of input token lengths for the planner in training (left) and test (right) data. The input consists of the query, top-k retrieved tools, planner-specific prompts, and execution history.

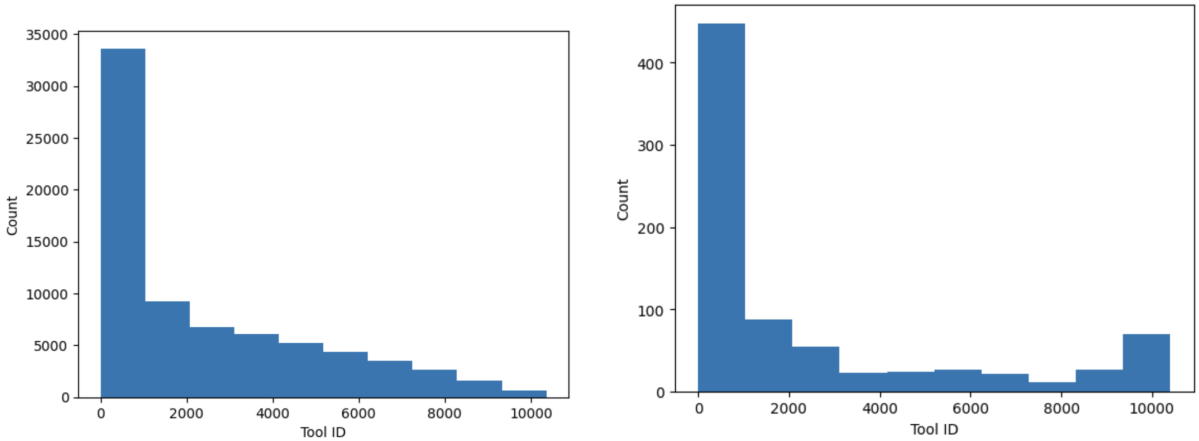

Figure 3: Frequency distributions of ground truth tools in training (left) and test (right) sets. Tool names have been converted to IDs for visualization clarity.

The ToolBench dataset was generated using ChatGPT. As is typical with LLM-generated data without access to additional knowledge, ToolBench is susceptible to hallucinations (Bang et al., 2023). An example of this can be seen in figure 4, where the synthetic dataset contains the hallucinated tool

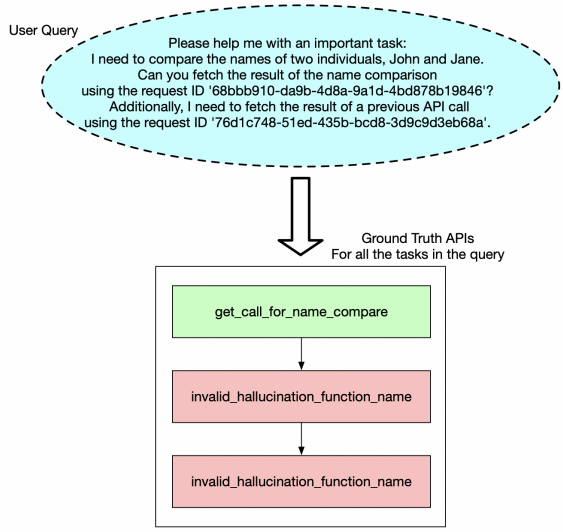

Figure 4: An example of tool hallucination in the ToolBench dataset.

invalid_hallucination_function_name at second and third steps. To address this issue, we removed requests with imaginary tool annotations, which are tools not included in the toolbox. Additionally, we manually reviewed and corrected remaining incorrectly extracted tools resulting from grammatical errors in the generated labels. Following the data cleaning process, the revised training and test datasets comprised of 25,489 and 274 complex requests, respectively. Using this dataset, we additionally performed TD using ChatGPT as described in section 3.1. After filtering out outliers with invalid tasks, we end up with a dataset size of 25,124 training data points and 268 test data points, which we use for all our experiments. The average number of subtasks in our final datasets is 2.934 (Std Dev. = 1.417) for the training set and 2.955 (Std Dev. = 1.39) for the test set.

3 Methodology

To evaluate the ProTIP framework for TR and planning tasks, we use both text-based and vector-based search baselines, as well as a strong baseline of TD-based planning, on the train and test splits from the ToolBench dataset after the preprocessing step to remove bad-quality data, as described in Section 2. Figure 5 illustrates the our envisioned end-to-end flow of ProTIP-based step-by-step planning.

3.1 Tool Retrieval

Tool Retrieval (TR) aims to identify the top-k most suitable tools to empower the planner to effectively execute all subtasks within a given complex request. The toolbox $\mathrm{T} = \{(t_1, d_1), (t_2, d_2), ..., (t_n, d_n)\}$ encompasses a collection of diverse tools t_i with predefined functionalities, captured in their tool descriptions d_i. A primary objective of TR is to extract subtasks from a complex query that align with the predefined functionalities of the tools, a concept we refer to as subtask-tool atomicity alignment and subsequently retrieve those tools.

When employing vector-based retrieval approaches, the toolbox T is typically represented as a vector database. An encoder $E_w(.)$ parameterized on w produces tool description embeddings, $E_w(d_i)$, which are then stored. Either the same or a query-specific encoder maps the complex query q into an embedding $E_w(q)$. A similarity measure, such as cosine similarity, between $E_w(d_i)$ and $E_w(q)$ is used to retrieve the top-k tools.

This study utilizes a comprehensive suite of retrieval methods, encompassing both pretrained and fine-tuned approaches, including our proposed ProTIP method, to evaluate the effectiveness of different TR strategies using the Recall@k metric.

BM25 The text-based retrieval baseline employed in this study is BM25 (Robertson et al., 1995). To retrieve the top-k tools, we utilize the full complex query q to compute BM25 scores for each tool description d_i.

Semantic Search For vector-based search, we utilize GTR-T5-XL (Ni et al., 2021) as the encoder for both query q and tool descriptions d_i. The cosine similarity measure $cos_sim(q, d_i)$ is employed to select the top-K most relevant tools.

Task Decomposition based TR Task Decomposition (TD) (Khot et al., 2022; Wu et al., 2023) breaks down a complex query q into a set of subqueries $\{q_1, q_2, ..., q_\tau\}$, where τ denotes the number of subtasks embedded within q, and each q_i represents a subquery corresponding to the ith subtask of q.

The evaluation of TD-based retrieval involves employing both BM25 and semantic search using GTR-T5-XL models. For each q_i from TD, we perform parallel retrieval using BM25 for text-based retrieval and GTR-T5-XL for vector-based retrieval. This results in the identification of top-k tools specific to each q_i. Subsequently, we employ an interleaving strategy to determine the final top-k tools for q. This approach of interleaving tools with TD serves as a straightforward yet powerful

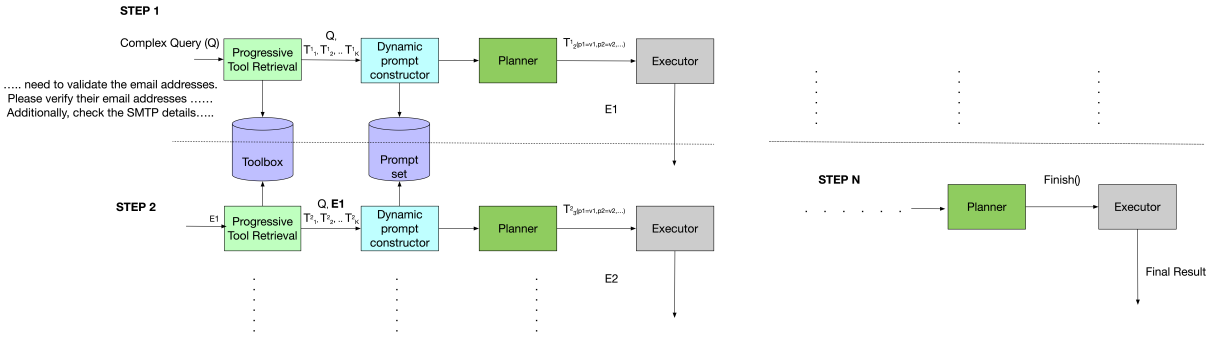

Figure 5: End-to-end processing of complex queries with ProTIP-based planning.

baseline. We opt for tool interleaving instead of directly utilizing all subqueries simultaneously, as the top-k tools obtained using the subquery set may not align with the desired top-k tools, where each subtask effectively covers relevant tools. We use the ChatGPT (OpenAI, 2023a) model to generate TD rewrites.

ProTIP We propose ProTIP, a progressive tool retrieval strategy, where top-k tools specific to each subtask are iteratively retrieved conditioned on execution history while retaining the subtask order. TD-based retrieval generates subtasks without factoring in the execution history. While TD-based retrieval can be adapted to leverage execution history, it still requires either an expensive pretrained LLM powerful enough to generate high-quality rewrites, or explicit task decomposition labels to fine-tune a lightweight model to generate rewrites. In addition, the TD labels should also ensure subtask-tool atomicity alignment is maintained.

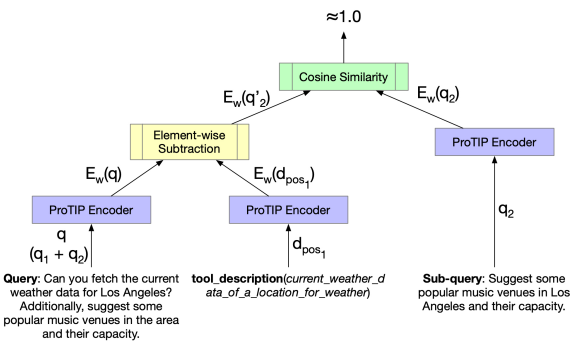

Figure 6: ProTIP's implicit learning mechanism for handling complex queries. Initial retrieval selects the tool relevant to the first subtask. Subsequently, the execution history, encompassing the tool description and query, is utilized to produce a resultant embedding $E(q_2')$. This embedding is specifically crafted to align with $E(q_2)$, which represents the second subtask, without the need for the subquery label q_2.

ProTIP is a lightweight retrieval model which does not require explicit labels. Instead of relying on intermediate text representation, ProTIP directly operates in the vector space by transforming input embeddings to eliminate subtasks which are already addressed.

Given a complex query q comprised of n subtasks, let $q_1...q_n$ denote queries that only capture each subtask from $1...n$. We use BERT-base-uncased* model as the encoder for both q and tool descriptions d_i, represented by $E_w(.)$. For each training batch of size b, we fine-tune $E_w(.)$ to always choose the ground-truth tool t_{pos_1} corresponding to subtask-1 by minimizing the distance between d_{pos_1} and q, while maximizing the distance between q and a set of randomly drawn negative examples, $T_{neg} = \{t_{neg_1}, t_{neg_2}, ..., t_{neg_{b-1}}\}$, from irrelevant tools. For subsequent retrieval steps, starting with subtask-2, we iteratively subtract $E_w(t_{pos_1})$ to $E_w(t_{pos_i})$ from $E_w(q)$ to arrive at an embedding that approximates a query $q^{'}$ that only represents subtasks from $i+1$ to n. This operation directly results in implicit learning of task decomposition while maintaining subtask-tool atomicity without the need for explicit labels, as depicted in Figure 6.

We use contrastive loss (Hadsell et al., 2006) to fine-tune our retrieval which is suited for metric-based learning. Given input pair with $I1$ and $I2$ inputs, contrastive loss encourages similar examples to be close, and dissimilar ones to have higher distance of at least margin m from each other. We define input $I1$ for query embedding as

$$I1 = \begin{cases} E_w(q), \text{for initial retrieval.} \\ E_w(q) - \sum_{1 \leq i < n}(E_w(d_i)), \text{otherwise;} \end{cases} \quad (1)$$

*https://huggingface.co/bert-base-uncased

where $\sum$ represents element-wise vector sum. We define tool description embedding* $I2$ as

$$I2 = E_w(d_{j+1}), \quad (2)$$

where $j \geq 0$.

The margin-based contrastive loss function is defined as

$$L = \frac{1}{2} l D^2 + \frac{1}{2}(1-l)\{\max(0, m-D)\}^2, \quad (3)$$

where l is a binary label which indicates whether the input pair consisting of the query $I1$ and tool description $I2$ embeddings is a positive ($l = 1$) or negative ($l = 0$) pair, m > 0 is the margin distance for dissimilar pairs and we use 0.3. D is a distance measure of choice and we use L2 norm between $I1$ and $I2$. Analogous to TD-based TR, we utilize a tool interleaving strategy to identify the final top-K tools for Recall@k evaluation.

3.2 Progressive Planning

To retrieve tools for the progressive planning task, we employ the tool retrieval (TR) strategies proposed in Section 3.1. We then perform supervised fine-tuning of the OpenLLaMA-7B-v2 model (Touvron et al., 2023; Geng and Liu, 2023; Computer, 2023), leveraging the HuggingFace Transformer library (Wolf et al., 2019). The model operates with a context window size of 2048 tokens. The prompt provided to the model consists of a fixed *instruction*, the *complex request*, and optional information such as the list of top-k *API candidates* (along with their metadata) and the *execution history*. This combination generates multiple distinct prompt variations.* In essence, our goal is to predict the next API to be executed in a multi-step plan given an input prompt containing the instruction, request, API candidates, and history. This requires unrolling the original data to form a sequence of prompts corresponding to each step in the plan.

Each interaction in the original data encompasses a natural language description of the *full query*. Additionally, each interaction comprises a total of p steps labeled *assistant* and f steps labeled *function*, along with potential inputs from the user labeled as *user* (we disregard *system* inputs). To prepare training and testing data for the planner, we unroll each interaction into p distinct unrolled data instances. Within each unrolled data instance, the text generated by the *assistant* for that specific step serves as the desired output, referred to as the *response*, while the entire sequence of steps preceding the current step constitutes the *history*. As a general rule, we utilize the original *full query* of the interaction as the *request*. In the event that an input occurs amidst the steps, we simply append it to the subsequent *request* segment. Notably, the training and test data differ in terms of the tools presented as *API candidates* in the input prompt.

Training: To provide the planner with a comprehensive set of potential tools, we utilize all p ground truth tools identified in the original data's *assistant* steps as *API candidates*. The oracle TR strategy employs the exact set of p ground truth tools ($p \leq 6$) required for the entire plan in the prompt for each step, closely resembling a memorization task. In contrast, top-k TR-based planners augment the prompt with an additional (K - p) randomly sampled tools for each step, where K > p, alongside the p ground truth tools. This approach introduces an element of uncertainty and challenges the planner to identify the most relevant tool for the current step. To ensure the correct tool is always present in the prompt, we maintain all ground truth tools from the full plan during the training of each step. This strategy guides the planner towards learning to select the most pertinent tool for the current task. Balancing between the LLM's maximum context window size of 2048 and the maximum value of p (6), we set k = 10 in our experiments. To prevent the LLM from exploiting the position of the correct tool, we randomize the order of the tools presented in the prompt during training and testing.

Testing: In the oracle TR strategy, we use exactly p ground truth tools identified from the original data's *assistant* steps as *API Candidates* for each step. This approach provides the Planner with a complete set of the necessary tools for each step, effectively making the task a tool selection problem. Conversely, for top-K TR-based planners, we utilize the top-10 tools retrieved by the corresponding algorithms, which may or may not include the ground truth tool. Additionally, we employ tool interleaving, where applicable.

Evaluation: While standard NLP metrics like Exact Match (EM) * and ROUGELSum (Lin, 2004)

*While we use tool descriptions, any information that helps predict the next tool could be used.

*Details in Appendix A.

*https://github.com/huggingface/

are commonly used to assess the overall text quality of the generated plan, our primary focus is on evaluating the LLM's performance in selecting appropriate tools. Therefore, we employ Tool Accuracy (TA) and Tool Hallucination (TH) metrics, specifically focusing on the API component of the predicted output and disregarding the parameter details.

4 Results

4.1 Tool Retrieval

For Recall@K, we start at K=6 given the maximum number of subtasks for a complex query in the ToolBench dataset is 6, as described in Section 2. Table 1 shows the recall of various retrieval approaches for different values of K.

Method	Recall@K			
	K=6	K=10	K=15	K=20
Full query based BM25	31.87	41	45.72	48.71
TD based BM25	41.26	47	50.70	54.74
Full query based SS	54.24	60.86	65.93	69.52
TD based SS	57.81	65.57	69.85	72.8
ProTIP	**80.55**	**81.36**	**82.35**	**83.48**

Table 1: Evaluation of various tool retrieval methods on the ToolBench test set. "TD-based" methods use task decomposition by ChatGPT and run retrieval in parallel for all subtasks, arriving at the top-K tools through interleaving. "SS" refers to Semantic Search.

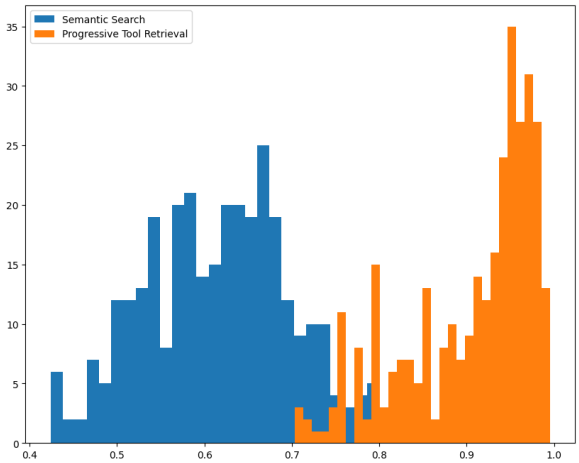

Figure 7: A comparison of cosine similarity distributions between Semantic Search and Progressive Tool Retrieval. Cosine similarity was computed between ground-truth tool descriptions and complex queries from the ToolBench test data.

Vector-based retrieval methods surpass text-based retrieval approaches, and TD-augmented retrieval employing an interleaving tools strategy demonstrates substantial gains over these baselines. ProTIP outperforms the best-performing TD-based retrieval method across all K values. As illustrated in Figure 7, ProTIP's utilization of contrastive learning enhances the cosine similarity between relevant tools and implicit subqueries. This improvement stems from iterative transformations performed directly in vector space, circumventing the requirement for intermediate text as in TD-based approaches. Consequently, ProTIP acquires implicit learning capabilities to predict the subsequent subtask and relevant tool while preserving the subtask order. The effectiveness of the ProTIP framework in handling queries characterized by complex language phenomena, such as disfluency, remains to be evaluated.

4.2 Progressive Planning

The progressive planning framework offers a multitude of configurable options, encompassing the prompt construction (with or without history, with or without candidate tools, etc.) and the type of tool metadata employed (e.g., tool name only versus tool name and description). To provide a representative assessment of the progressive planning task, we selected a subset of these configurations and evaluated the performance of various TR strategies on the preprocessed test set. The results are presented in Table 2. Settings 1-6 utilize various TR strategies introduced in this paper, while settings 7-10 employ the oracle TR strategy. To ensure a fair comparison with full-query-based TR strategies, we adopted the interleaving strategy (detailed in Section 3.1) for generating candidate tool sets for progressive TR (PTR).

Oracle Planner: To assess the performance of our proposed PTR-based fine-tuned planner, we establish a benchmark using several fine-tuned Oracle planners. These Oracle planners utilize the complete set of p ground truth (GT) tools, necessary for executing the entire plan, in the prompt, mimicking the Oracle TR algorithm. Setting 7-8 incorporates a total of 10 tools, comprising p GT tools and (10 - p) randomly sampled tools, while setting 9-10 employs precisely the p GT tools in the prompt. Setting 9-10 can be considered a strong upper bound achievable using Oracle TR for two reasons: a) the input prompt contains all GT tools required for ex-

---

evaluate

ID	Tool Retrieval Setting	Prompt	EM	RLSum	TA (%)	TH (%)
1	BM25 with full query	[T+H]	0.0442	0.3672	14.77	12.37
2	SS with full query	[T+H]	0.0619	0.4086	**21.72**	7.7
3	BM25 with TD query (Tool interleaving)	[T+H]	0.053	0.39	16.29	8.96
4	SS with TD query (Tool interleaving)	[T+H]	0.0833	0.4424	**25.88**	8.21
5	PTR (Tool interleaving)	[T]	0.0543	0.4129	19.82	2.02
6	PTR (Tool interleaving)	[T+H]	0.0896	0.4772	**36.49**	7.95
7	Oracle (GT + random tools)	[T]	0.0896	0.5232	44.57	4.17
8	Oracle (GT + random tools)	[T+H]	0.1805	0.6669	**77.53**	17.55
9	Oracle (only GT tools)	[T]	0.2146	0.579	46.59	5.3
10	Oracle (only GT tools)	[T+H]	0.3952	0.757	**80.3**	17.55

Table 2: Performance of progressive plan generation using various combinations of tool retrieval algorithms and prompt generation strategies. The prompt may comprise solely the list of tools ([T]) or both history and tools ([T+H]). We present the results for scenarios where the prompt includes only the tool name as tool metadata. For a given prompt setting (i.e., [T+H]), ProTIP consistently outperforms other baseline approaches, such as BM25 and SS, both with and without task decomposition. A substantial 15.79% absolute improvement in Recall@10 between TD-based SS and ProTIP translates to a 10.61% absolute increase in Tool Accuracy for the Planner, accompanied by a 0.26% reduction in Tool Hallucination.

ecuting the entire plan, and b) the fine-tuned model partially memorizes the tool selection process for each step given a specific query. We believe this represents an approximate upper bound on the performance attainable by a fine-tuned LLM-Planner employing the Oracle TR strategy, assuming the TR step achieves 100% Recall for the tools required for each step of the full query.

TR Planner: Consistently outperforming other baselines like BM25 and SS, PTR demonstrates superior performance under the [T+H] prompt setting, regardless of whether TD is employed. This superiority is further corroborated by the observed correlation between Recall@K of the TR algorithm (BM25 < SS < PTR) and Tool Accuracy (TA) of the Planner. Additionally, the better performance of BM25 and SS-based TR for task-decomposed queries is reflected in the corresponding Planner performance. This aligns with the Planner's design, which mandates tool selection from the TR algorithm's retrieved set. Interestingly, the Tool Hallucination (TH) percentage, which represents the proportion of times the Planner creates non-existent tools, reveals a consequence of failing to adhere to this design principle. PTR without history exhibits the lowest TH percentage, despite its relatively low TA. Upon incorporating history, both PTR (setting 6) and Oracle (settings 8 and 10) experience an increase in TA and TH, potentially due to truncation issues (discussed in Section 5). Notably, higher TA in PTR leads to marginal improvements in Exact Match (EM) and ROUGELSum (RLSum), metrics that evaluate the entire predicted output, including tool parameters. The relatively modest gains in these metrics suggest that further performance enhancements can be achieved by focusing on tool parameter optimization. The performance gap between Oracle planners (settings 6 to 10) and the PTR-based planner highlights the potential for further Planner performance improvements.

Importance of history for Planning: The inclusion of the history of previous steps demonstrates a significant performance boost in planning across all settings. This improvement is particularly pronounced for both Oracle-based planning (approx. 70+% improvement between settings 9 and 10) and PTR-based planning (approx. 80+% improvement between settings 5 and 6) in TA. Intuitively, incorporating history is crucial as it can aid in selecting the most appropriate tool, especially during branching scenarios that may arise during the execution of the previous tool (e.g., if the previous tool executed successfully, select Tool-A; otherwise, select Tool-B). However, incorporating history into the prompt raises concerns about truncation due to the increased token count (more details in Section 5).

5 Limitations and Discussion

Due to the computational demands of hyperparameter tuning, we were unable to optimize the settings for all configurations. Each configuration requires 8 A100 GPUs on AWS, resulting in significant

time and resource consumption. Consequently, we focused our hyperparameter tuning efforts on the ProTIP (settings 5 and 6) and Oracle (settings 9 and 10). The detailed hyperparameter values for all settings in Table 2 are provided in Appendix B. To ensure a fair comparison with the full query-based TR strategies, we employed the interleaving strategy (described in Section 3.1) for generating candidate tool sets for PTR. We recognize that this approach is not ideal for evaluating the planner's performance under PTR, as the optimal approach would involve retrieving tools step-by-step and performing planning at each step. However, this would require a mechanism to execute the predicted API call at each step and incorporate the resulting output into the planning process for the next step. While we are currently investigating potential solutions for this challenge, planner design is not the primary focus of this work. Therefore, we defer the development and evaluation of end-to-end step-by-step planning experiments, including performance tuning, to future research.

The experiment results reveal a substantial performance gap between the Oracle planner and the TR-based planners. This disparity can be attributed to two key factors. First, the Oracle planner (settings 9 and 10) utilizes the exact set of p ground truth tools specified in the prompt for each progressive planning step ($p \leq 6$), whereas the TR planners operate on a larger candidate set of K=10 tools. This restricted tool selection for the Oracle planner (settings 9 and 10) likely contributes to its improved performance. This observation is further supported by the higher TA achieved in setting 10 (using exactly p ground truth tools) compared to setting 8 (using K tools, with p ground truth tools and (10 - p) randomly sampled tools).

The tool distribution discrepancy between the train and test sets, particularly for tool IDs greater than 8000, as evident in Figure 3, may partially explain the inferior performance of all retrieval-based planners. This disparity in tool distribution could hinder the effectiveness of the TR strategies, potentially leading to suboptimal planning decisions, unless tool metadata is further enriched and included in the prompt during training to support for better generalization. Additionally, we observed a poor Accuracy for the special *Finish* tool, which resulted in overall low performance in all the TR settings.

The training strategy of employing the p ground truth tools alongside (K - p) randomly sampled tools in the prompt may contribute to the lower performance of the TR planner models. The presence of the ground truth tools alongside semantically dissimilar randomly sampled tools during training likely facilitates the models' ability to identify the correct tool. However, during testing, the prompt comprises top-K tools retrieved by the TR algorithms, which may exhibit semantic similarity to the ground truth tool. This semantic similarity poses a challenge for the models during inference, leading to the observed low TA values for all TR-based planner models. Utilizing the top-K tools retrieved by the TR algorithms during training could exacerbate this issue, as there is a risk of the prompt not containing the correct tool for the corresponding step. This would further complicate the learning process for the LLM and increase the likelihood of hallucinations.

To address this limitation, in future, an alternative training data creation strategy could be employed. Instead of using randomly sampled tools, the training data could incorporate tools retrieved by the TR algorithm on the training set. Additionally, to ensure that the training process effectively addresses all steps, the ground truth tool for the current step could be injected into the prompt if it is not already present. By adopting this modified training approach, we aim to enhance the performance of the TR planner models and improve their generalization capabilities. The Instructions part of the prompt are generic and can be further modified (i.e., made more precise for each scenario) to be more specific to various prompt settings. Also, we did not conduct an exhaustive study to test the robustness of the planner output to different types of input prompt variations (e.g.: paraphrased query as inputs, semantically similar tools, unseen tools in the test set etc.), which we leave as future work.

Our experiments highlight the significance of the *history* in achieving higher TA for both Oracle (setting 9 vs. 10) and PTR (setting 5 vs. 6) based planning strategies. However, we believe that TA can be further improved while reducing TH, particularly for TR planners with K=10 tools, as the *history* contributes to the long context issue. We observe that for the scenarios where the input size becomes close to the maximum context window size, the LLM could generate empty plans, which contributes to 3% to 5% of the errors in our experiments, thereby negatively impacting the TA. Note that the original data contains intermediate

steps with verbose outputs that provide minimal contextual information (e.g., weather API outputs with latitude, longitude, last updated time, etc.), all of which may not be essential for determining the next API. Preserving these verbose outputs in the *history* exacerbates the truncation problem, thereby negatively impacting the learning and plan generation capability of the model. This issue can be further aggravated by incorporating more tool metadata (tool description, parameters, API signature, etc.) into the prompt, which will increase the input length of the prompt even further. However for better generalization to unseen tools, ideally we want to incorporate such additional metadata into the prompt, which requires further investigation.

Increasing the context window size of LLMs (e.g., to 4096 or higher) or employing techniques that allow for larger input text (e.g., as proposed in (Beltagy et al., 2020)) can largely alleviate the truncation problem. However, even with large context window, studies by (Liu et al., 2023) indicate that LLMs tend to focus on information at the beginning or end of the prompt, even with a large context window. Therefore, alongside exploring LLMs with larger context windows, there is a need to develop methods for effectively compressing and presenting relevant contextual information, particularly the *history*, to the LLM (Ge et al., 2023) to enhance performance.

In the current work, we focused heavily on the tool accuracy across the tool retrieval and planning steps (Patil et al., 2023). Tool parameter accuracy is another important aspect of the planner output (Shen et al., 2023b), which requires further investigations to improve the performance. We did not conduct any safety study or red-teaming to evaluate the bias or risks emanating from the outputs generated by the fine-tuned LLM. We want to refer to a contemporary work by (Valmeekam et al., 2023) which has pointed out that the ability of LLM's to generate "executable plans" in a completely autonomous way is very limited. In our work, while planning is not the primary focus, we observed that plan generation using supervised fine-tuning of a LLM is not an easy task, specifically with a relatively small LLM (e.g.: LLM with 7B parameters). We believe further research is required to enhance our understanding of the true capabilities of LLM's for the planning task.

6 Related Work

Tool Retrieval using Neural embedding: Vector databases enable storing tool name and description embeddings generated by an encoding model (Cer et al., 2018). These embeddings are then leveraged for semantic similarity computation, utilizing measures like cosine similarity, with queries or sub-queries. Building on the established approach of utilizing neural networks to generate task-specific semantic word/sentence embeddings for information retrieval and NLP tasks (Zhang et al., 2016), this work proposes a tool embedding generation strategy specifically designed to facilitate step-by-step planning.

Word embeddings (Mikolov et al., 2013b; Pennington et al., 2014; Levy and Goldberg, 2014; Jiao and Zhang, 2021), learned vectors representing various linguistic and semantic aspects of words, have revolutionized Natural Language Processing by enabling efficient solutions to diverse tasks like analogy queries (Levy and Goldberg, 2014; Allen and Hospedales, 2019). Building upon this success, research has extended to generating sentence, paragraph, and document-level embeddings (Le and Mikolov, 2014; Wieting et al., 2015; Li et al., 2015) for various applications. Similarly, the Knowledge Graph domain utilizes node embedding to encode entity relationships, trained with custom objective functions to capture latent relationships in the vector space for subsequent exploitation (Wang et al., 2017). We leverage this paradigm, employing progressive tool retrieval with fine-tuned embeddings optimized for the step-by-step planning task.

LLM as Planner: LLMs have emerged as potent few-shot learners (Brown et al., 2020; Rae et al., 2022), exhibiting remarkable prowess across diverse language tasks. However, planning remains a challenging research frontier despite their impressive performance in these domains. Planning involves decomposing a high-level natural language (NL) task into a sequence of executable steps realizable by an agent. This process demands both language comprehension and an understanding of a predefined set of actions, tools, APIs, and other grounding elements. In the realm of embodied agents, LLMs have been harnessed to decompose NL instructions into simpler, more manageable units (Huang et al., 2022; Ahn et al., 2022; Singh et al., 2022; Khot et al., 2022; Wu et al., 2023; Shen et al., 2023b). Notably, using LLMs to generate tool/API calls as part of the planning process

can be akin to multi-step program synthesis (Li et al., 2023; Nijkamp et al., 2022, 2023). More recent works have tried to further improve LLM performance by adding the capability to reason/-criticize the LLM outputs (Kim et al., 2023; Yao et al., 2022).

While contemporary research has emphasized leveraging tools to enhance LLM capabilities, most existing retrieval systems rely on vector databases, similar to the renowned Retrieval Augmented Generation (RAG) technique (Lewis et al., 2021), to store and retrieve non-parametric knowledge absent in the LLM. Recent work has explored individual tools like web search engines (Nakano et al., 2022), calculators (Andor et al., 2019), and generic toolsets (Schick et al., 2023) for planning, while others have integrated LLMs with expansive API collections to address more open-ended tasks (Patil et al., 2023; Shen et al., 2023a; Liang et al., 2023; Qin et al., 2023b). Fine-tuning with tool-specific data is often employed to improve task performance. However, as the number of tool grows, retrieval-based systems emerge as an efficient means for selecting the relevant tools for a given request (Patil et al., 2023; Qin et al., 2023b). Building upon this paradigm, our approach proposes the novel concept of incrementally generating tool candidates specific to the current sub-step within a multi-step planning task, ultimately enhancing the LLM's overall planning performance.

7 Conclusion

We introduce ProTIP, a novel lightweight tool retrieval framework, which surpasses LLM-based planning agents equipped with state-of-the-art task decomposition retrieval powered by ChatGPT. ProTIP's iterative vector transformations, enabled by contrastive learning, facilitate implicit learning of sequential subtask prediction, eliminating the need for explicit subtask labels. Additionally, ProTIP effectively handles "subtask-tool atomicity alignment." On the ToolBench dataset, ProTIP framework surpasses the ChatGPT task decomposition based approach by 24% on Recall@K=10 for Tool Retrieval and by 41% on Tool Accuracy for plan generation.

8 Acknowledgements

We would like to thank Stephen Pulman, Russ Webb and Arturo Argueta for their valuable feedback. Also we thank Jiarui Lu, Yuan Zhang, Xuan Wang, Hans Han, and Jian Zhang for providing infrastructure support to fine-tune LLMs.

References

Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Daniel Ho, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Eric Jang, Rosario Jauregui Ruano, Kyle Jeffrey, Sally Jesmonth, Nikhil J Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Kuang-Huei Lee, Sergey Levine, Yao Lu, Linda Luu, Carolina Parada, Peter Pastor, Jornell Quiambao, Kanishka Rao, Jarek Rettinghouse, Diego Reyes, Pierre Sermanet, Nicolas Sievers, Clayton Tan, Alexander Toshev, Vincent Vanhoucke, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Mengyuan Yan, and Andy Zeng. 2022. Do as i can, not as i say: Grounding language in robotic affordances.

Carl Allen and Timothy Hospedales. 2019. Analogies explained: Towards understanding word embeddings.

Daniel Andor, Luheng He, Kenton Lee, and Emily Pitler. 2019. Giving bert a calculator: Finding operations and arguments with reading comprehension.

Yejin Bang, Samuel Cahyawijaya, Nayeon Lee, Wenliang Dai, Dan Su, Bryan Wilie, Holy Lovenia, Ziwei Ji, Tiezheng Yu, Willy Chung, et al. 2023. A multitask, multilingual, multimodal evaluation of chatgpt on reasoning, hallucination, and interactivity. *arXiv preprint arXiv:2302.04023*.

Iz Beltagy, Matthew E Peters, and Arman Cohan. 2020. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901.

Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott Lundberg, et al. 2023. Sparks of artificial general intelligence: Early experiments with gpt-4. *arXiv preprint arXiv:2303.12712*.

Daniel Cer, Yinfei Yang, Sheng yi Kong, Nan Hua, Nicole Limtiaco, Rhomni St. John, Noah Constant, Mario Guajardo-Cespedes, Steve Yuan, Chris Tar, Yun-Hsuan Sung, Brian Strope, and Ray Kurzweil. 2018. Universal sentence encoder.

Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion

Stoica, and Eric P. Xing. 2023. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality.

Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. 2022. Palm: Scaling language modeling with pathways. *arXiv preprint arXiv:2204.02311*.

Together Computer. 2023. Redpajama-data: An open source recipe to reproduce llama training dataset.

Tao Ge, Jing Hu, Xun Wang, Si-Qing Chen, and Furu Wei. 2023. In-context autoencoder for context compression in a large language model. *arXiv preprint arXiv:2307.06945*.

Xinyang Geng and Hao Liu. 2023. Openllama: An open reproduction of llama.

R. Hadsell, S. Chopra, and Y. LeCun. 2006. Dimensionality reduction by learning an invariant mapping. In *Proceedings of 2006 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'06)*.

Wenlong Huang, Pieter Abbeel, Deepak Pathak, and Igor Mordatch. 2022. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents.

Qilu Jiao and Shunyao Zhang. 2021. A brief survey of word embedding and its recent development. In *2021 IEEE 5th Advanced Information Technology, Electronic and Automation Control Conference (IAEAC)*, volume 5, pages 1697–1701. IEEE.

Tushar Khot, Harsh Trivedi, Matthew Finlayson, Yao Fu, Kyle Richardson, Peter Clark, and Ashish Sabharwal. 2022. Decomposed prompting: A modular approach for solving complex tasks. In *Proceedings of the Eleventh International Conference on Learning Representations*.

Geunwoo Kim, Pierre Baldi, and Stephen McAleer. 2023. Language models can solve computer tasks. *arXiv preprint arXiv:2303.17491*.

Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large language models are zero-shot reasoners. *Advances in neural information processing systems*, 35:22199–22213.

Quoc Le and Tomas Mikolov. 2014. Distributed representations of sentences and documents. In *International conference on machine learning*, pages 1188–1196. PMLR.

Omer Levy and Yoav Goldberg. 2014. Linguistic regularities in sparse and explicit word representations. In *Proceedings of the eighteenth conference on computational natural language learning*, pages 171–180.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2021. Retrieval-augmented generation for knowledge-intensive nlp tasks.

Jiwei Li, Minh-Thang Luong, and Dan Jurafsky. 2015. A hierarchical neural autoencoder for paragraphs and documents. *arXiv preprint arXiv:1506.01057*.

Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. 2023. Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161*.

Yaobo Liang, Chenfei Wu, Ting Song, Wenshan Wu, Yan Xia, Yu Liu, Yang Ou, Shuai Lu, Lei Ji, Shaoguang Mao, Yun Wang, Linjun Shou, Ming Gong, and Nan Duan. 2023. Taskmatrix.ai: Completing tasks by connecting foundation models with millions of apis.

Chin-Yew Lin. 2004. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pages 74–81, Barcelona, Spain. Association for Computational Linguistics.

Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2023. Lost in the middle: How language models use long contexts. *arXiv preprint arXiv:2307.03172*.

Tomáš Mikolov, Wen-tau Yih, and Geoffrey Zweig. 2013a. Linguistic regularities in continuous space word representations. In *Proceedings of the 2013 conference of the north american chapter of the association for computational linguistics: Human language technologies*, pages 746–751.

Tomas Mikolov, Wen-tau Yih, and Geoffrey Zweig. 2013b. Linguistic regularities in continuous space word representations. In *Proceedings of the 2013 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 746–751, Atlanta, Georgia. Association for Computational Linguistics.

Sewon Min, Xinxi Lyu, Ari Holtzman, Mikel Artetxe, Mike Lewis, Hannaneh Hajishirzi, and Luke Zettlemoyer. 2022. Rethinking the role of demonstrations: What makes in-context learning work? In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*.

Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, Xu Jiang, Karl Cobbe, Tyna Eloundou, Gretchen Krueger, Kevin Button, Matthew Knight, Benjamin Chess, and John Schulman. 2022. Webgpt: Browser-assisted question-answering with human feedback.

Jianmo Ni, Chen Qu, Jing Lu, Zhuyun Dai, Gustavo Hernández Ábrego, Ji Ma, Vincent Y. Zhao, Yi Luan, Keith B. Hall, Ming-Wei Chang, and Yinfei Yang. 2021. Large dual encoders are generalizable retrievers.

Erik Nijkamp, Hiroaki Hayashi, Caiming Xiong, Silvio Savarese, and Yingbo Zhou. 2023. Codegen2: Lessons for training llms on programming and natural languages. *arXiv preprint arXiv:2305.02309*.

Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2022. Codegen: An open large language model for code with multi-turn program synthesis. *arXiv preprint arXiv:2203.13474*.

OpenAI. 2023a. Introducing chatgpt.

R OpenAI. 2023b. Gpt-4 technical report. arxiv 2303.08774. *View in Article*, 2.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744.

Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. 2023. Gorilla: Large language model connected with massive apis. *arXiv preprint arXiv:2305.15334*.

Jeffrey Pennington, Richard Socher, and Christopher D Manning. 2014. Glove: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pages 1532–1543.

Yujia Qin, Shengding Hu, Yankai Lin, Weize Chen, Ning Ding, Ganqu Cui, Zheni Zeng, Yufei Huang, Chaojun Xiao, Chi Han, et al. 2023a. Tool learning with foundation models. *arXiv preprint arXiv:2304.08354*.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2023b. Toolllm: Facilitating large language models to master 16000+ real-world apis.

Jack W. Rae, Sebastian Borgeaud, Trevor Cai, Katie Millican, Jordan Hoffmann, Francis Song, John Aslanides, Sarah Henderson, Roman Ring, Susannah Young, Eliza Rutherford, Tom Hennigan, Jacob Menick, Albin Cassirer, Richard Powell, George van den Driessche, Lisa Anne Hendricks, Maribeth Rauh, Po-Sen Huang, Amelia Glaese, Johannes Welbl, Sumanth Dathathri, Saffron Huang, Jonathan Uesato, John Mellor, Irina Higgins, Antonia Creswell, Nat McAleese, Amy Wu, Erich Elsen, Siddhant Jayakumar, Elena Buchatskaya, David Budden, Esme Sutherland, Karen Simonyan, Michela Paganini, Laurent Sifre, Lena Martens, Xiang Lorraine Li, Adhiguna Kuncoro, Aida Nematzadeh, Elena Gribovskaya, Domenic Donato, Angeliki Lazaridou, Arthur Mensch, Jean-Baptiste Lespiau, Maria Tsimpoukelli, Nikolai Grigorev, Doug Fritz, Thibault Sottiaux, Mantas Pajarskas, Toby Pohlen, Zhitao Gong, Daniel Toyama, Cyprien de Masson d'Autume, Yujia Li, Tayfun Terzi, Vladimir Mikulik, Igor Babuschkin, Aidan Clark, Diego de Las Casas, Aurelia Guy, Chris Jones, James Bradbury, Matthew Johnson, Blake Hechtman, Laura Weidinger, Iason Gabriel, William Isaac, Ed Lockhart, Simon Osindero, Laura Rimell, Chris Dyer, Oriol Vinyals, Kareem Ayoub, Jeff Stanway, Lorrayne Bennett, Demis Hassabis, Koray Kavukcuoglu, and Geoffrey Irving. 2022. Scaling language models: Methods, analysis & insights from training gopher.

Stephen Robertson, S. Walker, S. Jones, M. M. Hancock-Beaulieu, and M. Gatford. 1995. Okapi at trec-3. In *Overview of the Third Text REtrieval Conference (TREC-3)*, pages 109–126. Gaithersburg, MD: NIST.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools.

Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. 2023a. Hugginggpt: Solving ai tasks with chatgpt and its friends in huggingface. *arXiv preprint arXiv:2303.17580*.

Yongliang Shen, Kaitao Song, Xu Tan, Wenqi Zhang, Kan Ren, Siyu Yuan, Weiming Lu, Dongsheng Li, and Yueting Zhuang. 2023b. Taskbench: Benchmarking large language models for task automation. *arXiv preprint arXiv:2311.18760*.

Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, and Animesh Garg. 2022. Progprompt: Generating situated robot task plans using large language models.

Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. Stanford alpaca: An instruction-following llama model. https://github.com/tatsu-lab/stanford_alpaca.

Romal Thoppilan, Daniel De Freitas, Jamie Hall, Noam Shazeer, Apoorv Kulshreshtha, Heng-Tze Cheng, Alicia Jin, Taylor Bos, Leslie Baker, Yu Du, et al. 2022. Lamda: Language models for dialog applications. *arXiv preprint arXiv:2201.08239*.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.

Karthik Valmeekam, Sarath Sreedharan, Matthew Marquez, Alberto Olmo, and Subbarao Kambhampati. 2023. On the planning abilities of large language models (a critical investigation with a proposed benchmark). *arXiv preprint arXiv:2302.06706*.

Quan Wang, Zhendong Mao, Bin Wang, and Li Guo. 2017. Knowledge graph embedding: A survey of approaches and applications. *IEEE Transactions on Knowledge and Data Engineering*, 29(12):2724–2743.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2022. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*.

Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, et al. 2022a. Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682*.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022b. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837.

John Wieting, Mohit Bansal, Kevin Gimpel, and Karen Livescu. 2015. Towards universal paraphrastic sentence embeddings. *arXiv preprint arXiv:1511.08198*.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. 2019. Huggingface's transformers: State-of-the-art natural language processing.

Yue Wu, So Yeon Min, Yonatan Bisk, Ruslan Salakhutdinov, Amos Azaria, Yuanzhi Li, Tom Mitchell, and Shrimai Prabhumoye. 2023. Plan, eliminate, and track — language models are good teachers for embodied agents.

Sang Michael Xie, Aditi Raghunathan, Percy Liang, and Tengyu Ma. 2021. An explanation of in-context learning as implicit bayesian inference. *arXiv preprint arXiv:2111.02080*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*.

Aohan Zeng, Xiao Liu, Zhengxiao Du, Zihan Wang, Hanyu Lai, Ming Ding, Zhuoyi Yang, Yifan Xu, Wendi Zheng, Xiao Xia, et al. 2022. Glm-130b: An open bilingual pre-trained model. *arXiv preprint arXiv:2210.02414*.

Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. 2022. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*.

Ye Zhang, Md Mustafizur Rahman, Alex Braylan, Brandon Dang, Heng-Lu Chang, Henna Kim, Quinten McNamara, Aaron Angert, Edward Banner, Vivek Khetan, et al. 2016. Neural information retrieval: A literature review. *arXiv preprint arXiv:1611.06792*.

A Prompt Variations

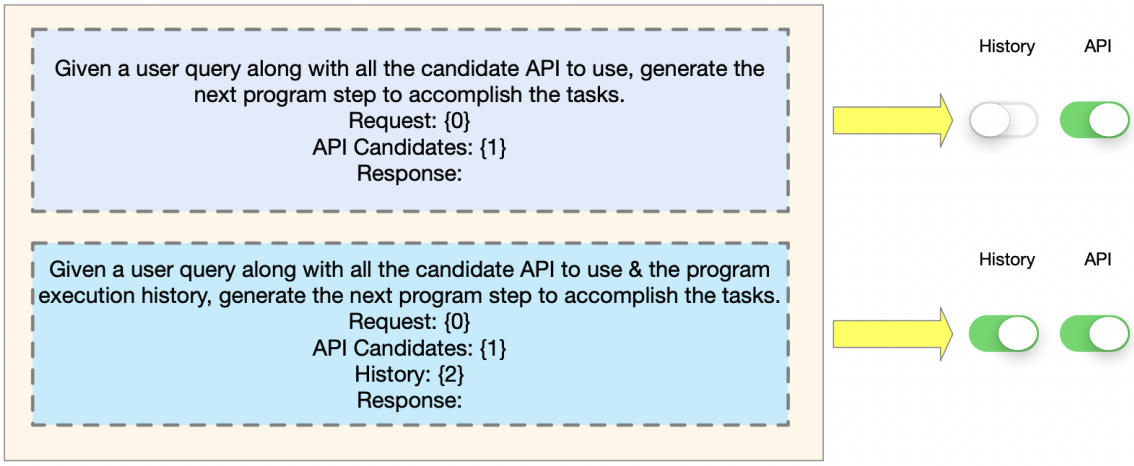

Figure 8: Prompt variants for plan generation with and without execution history.

B Hyperparemeters

Due to time and resource constraints, we were unable to perform hyperparameter tuning for all experimental setups. The following table lists the hyperparameters used for each of the settings.

Table 3: Hyperparameters for training

Setting ID	Epochs	LR	Batch	Warmup Steps
1	3	3e-5	64	20
2	3	3e-5	64	20
3	3	3e-5	64	20
4	3	3e-5	64	20
5	3	3e-5	64	20
6	3	5e-5	128	20
7	3	5e-5	128	20
8	3	5e-5	128	20
9	3	2e-5	64	20
10	3	2e-5	64	20

This model utilizes a maximum context window size of 2048 and employs a weight decay of 0 across all experiments.